

CS 431: Computer and Network Security

Assignment 1

columns-101 — python3 solution.py 1

Flag : THENUMBEROFRAILSFORTHERAILFENCECIPHERIS16

The problem provides a ciphertext encrypted via [columnar transposition](#) with a known key length of 5. While [dcode.fr](#) was used for initial analysis, the solution was implemented from scratch. We brute-forced all $5! = 120$ column permutations, then arranged the ciphertext in the columns and scored the resulting plaintexts (read row wise in column order given by the key) using English trigram frequency analysis (based on [english_3grams.csv](#) and using log likelihood). Note that the ciphertext length (41) is not divisible by key length (5). So one column would have had an extra row. This was also accounted for while bruteforcing. The permutation with the highest score yielded the readable flag, which provided the hint for the subsequent Rail Fence problem.

vigenere-101 — python3 solution.py 2

Flag : CNS431{v1g3n3r3_c1ph3r_1s_c14ss1c}

The problem title tells us that [vigenere cipher](#) is used to encrypt the ciphertext. We run the given command, `nc 10.0.118.104 5050`, which gives us the ciphertext and the key “VICTORY”. Note that the ciphertext contains numbers and exclamation marks as well. To decrypt, we repeat the key to match the length of the ciphertext (skipping over the non-alphabets), and then for every character, go to the corresponding row of the [vigenere table](#) (for any character, the corresponding row in the vigenere table starts the alphabet sequence from that character and loops around after Z), find the label of the column containing the corresponding ciphertext character. We keep the non-alphabets as it is. Decoding the ciphertext, we get `Gw3atJob!Y0uCr4ck3dIt`. We submit this to the server, and the server responds with the flag: `CNS431{v1g3n3r3_c1ph3r_1s_c14ss1c}`. At the time of solving, We use [dcode.fr](#) for this. However while writing the writeup, we wrote the decrypt function from scratch. Also, we include the communication with the server in our script. It handles [interactive input/output](#) and [printing the flag](#).

Rail Force One — python3 solution.py 3

Flag : MAYBETRYRAILS4AFTERTHISCNMEASHIKWFUPVRLZGEUB00JEOTEYOQRXDHD

The problem title and the previous flag (“...RAILFENCECIPHERIS16”) suggested a [Rail Fence cipher](#) with 16 rails. We solved this initially using [dcode.fr](#). We then wrote a custom Rail Fence decoder in Python. The script reconstructs the zigzag matrix pattern for 16 rails, fills it with the ciphertext, and reads off the plaintext row by row. Note that the matrix will be of the shape (rails $\times$ length of ciphertext). This successfully decrypted the flag, which contained a hint for the next challenge.

Rail Force Two — python3 solution.py 4

Flag : THEQUICKBROWNFOXJUMPEDOVERTHELAZYDOGS

The plaintext from *Rail Force One* started with the hint “MAYBETRYRAILS4AFTERTHIS”. Following the problem description, we stripped these first 23 characters to isolate the nested ciphertext. Using the hint “RAILS 4”, we applied our Rail Fence decoder with 4 rails (again, verified with [dcode.fr](#) but solved via script). This revealed the final plaintext flag.

```

nment 1$ python3 solution.py 1
Problem: columns-101
THENUMBEROFRAILSFORTHERAILFENCECIPHERIS16 | -144.52
THNEUMBREOFRIALSFROTHEARILFNECECPIHERSI16 | -161.33
TNEHUMREBOFIARLSROFTHAREILNEFCEPICHERSIR16 | -163.62
TEHNUMBROFARILSOFRTHREAILFENCEICPHEIRS16 | -164.89
TENHUMERBOFAIRLSORFTHRAEILNEFCEIPCHEISR16 | -167.96
GUTHENOMBERLFRATTSFORIHERACLNFENHECIP1ERIS | -168.46
EHNUMBROFARILSOFRTHREAILFENCEICPHEIRS16T | -171.37

nment 1$ python3 solution.py 2
Problem: vigenere-101
Decoded vigenere: Gw3atJob!Y0uCr4ck3dIt
Flag: CNS431{v1g3n3r3_c1ph3r_1s_c14ss1c}

ts/Assignment 1$ python3 solution.py 3
Problem: Rail Force One
M A Y B E T R Y R A I L S 4 A F T E R T H I S C N M E A S H I K W F U P V R L Z G E U B 0 0 J E O T E Y O Q R X D H D
H S I D
A K W
E F
N U
C P
T V
I S
L Z
S R
4 E
A T
F
U B O
O
Flag: MAYBETRYRAILS4AFTERTHISCNMEASHIKWFUPVRLZGEUB00JEOTEYOQRXDHD
jsm@LAPTOP-GS8HQLEP: /mnt/c:/Users/JSKRAJA/Desktop
nment 1$ python3 solution.py 4
Problem: Rail Force Two
T C N M E A S
H I K W F U P V R L Z G
E U B O O J E O T E Y O
Q R X D H D
Flag: THEQUICKBROWNFOXJUMPEDOVERTHELAZYDOGS

```

Source Code: solution.py

```
import sys
import subprocess
import math
from itertools import permutations

def problem1():
    """Reference:
    https://www.geeksforgeeks.org/dsa/columnar-transposition-cipher/
    3grams to score the results:
    https://calmcode.io/datasets/english_3grams
    """
    print("Problem: columns-101")
    ciphertext = "UOLTICH1EEAOREIITMFSHLEE6NRIRANPSHBRFEFCR"
    len_k = 5
    sols = []
    col_len, rem = divmod(len(ciphertext), len_k)
    trigram_counts = {}
    total_count = 0
    with open("english_3grams.csv", "r", encoding="utf-8") as f:
        for line in f:
            parts = line.strip().split(",")
            if len(parts) == 2:
                gram = parts[0].upper()
                count = int(parts[1])
                trigram_counts[gram] = count
                total_count += count

    def scorer(text):
        score = 0
        text = text.upper()
        for i in range(len(text) - 2):
            trigram = text[i : i + 3]
            if trigram in trigram_counts:
                prob = trigram_counts[trigram] / total_count
                score += math.log10(prob)
            else:
                score += math.log10(1 / (total_count * 10))

        return score

    for perm in permutations(list(range(1, 1 + len_k))):
        matrix_cols = [""] * (len_k + 1)
        idx = 0
        for k in perm:
            col_height = col_len + 1 if k <= rem else col_len
            chunk = ciphertext[idx : idx + col_height]
            matrix_cols[k] = chunk
            idx += col_height
        decoded = []
        for r in range(col_len + 1):
            for c in range(1, len_k + 1):
                if r < len(matrix_cols[c]):
                    decoded.append(matrix_cols[c][r])

        decoded_msg = "".join(decoded)
        sols.append((perm, decoded_msg, scorer(decoded_msg)))

    sols.sort(key=lambda x: x[2], reverse=True)
    print("\n".join(map(lambda x: f"{x[1]} | {x[2]:4.2f}", sols[:7])))
```

```

def problem2():
    """References:
    https://www.geeksforgeeks.org/dsa/vigenere-cipher/
    https://stackoverflow.com/questions/19880190/interactive-input-output-using-python
    https://stackoverflow.com/questions/59787040/stuck-in-infinite-loop-while-trying-to-read-all-lines-in-proc-stdout-readline
    """

    print("Problem: vigenere-101")
    key = "VICTORY"
    len_k = len(key)
    ciphertext = "Be3cmXfz!T0cEk4qb3bDb"
    alphabets = "abcdefghijklmnopqrstuvwxyz"

    def char2idx(char):
        return ord(char.lower()) - ord("a")

    def decode(c1, c2):
        rotated = alphabets[char2idx(c2) :] + alphabets[: char2idx(c2)]
        dec = alphabets[rotated.find(c1.lower())]
        return dec if not c1.isupper() else dec.upper()

    decoded = ""
    cnt = 0
    for idx in range(len(ciphertext)):
        if ciphertext[idx].isalpha():
            decoded += decode(ciphertext[idx], key[cnt % len_k])
            cnt += 1
        else:
            decoded += ciphertext[idx]

    print(f"Decoded vigenere: {decoded}")
    proc = subprocess.Popen(
        ["nc", "10.0.118.104", "5050"],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        universal_newlines=True,
    )
    lines = []
    for _line in iter(proc.stdout.readline, b""):
        if _line == "\n":
            break
        lines.append(_line)
    proc.stdin.write(f"{decoded.strip()}\n")
    proc.stdin.flush()
    lines = []
    for _line in iter(proc.stdout.readline, b""):
        lines.append(_line)
        if _line.endswith("\n"):
            break
    print("Flag:", lines[0].split(": ")[-1])

def rail_fence_decode(rails, ciphertext):
    """Reference:
    https://www.geeksforgeeks.org/dsa/rail-fence-cipher-encryption-decryption/
    """
    cols = len(ciphertext)
    m = [{" " for _ in range(cols)} for _ in range(rails)]

```

```

r, c = 0, 0
for _ in range(cols):
    if r == 0:
        down = True
    elif r == rails - 1:
        down = False
    m[r][c] = "t"
    c += 1
    if down:
        r += 1
    else:
        r -= 1
# print('\n'.join([''.join(rr) for rr in m]))
idx = 0

for r in range(rails):
    for c in range(cols):
        if m[r][c] == "t":
            m[r][c] = ciphertext[idx]
            idx += 1

print('\n'.join([''.join(rr) for rr in m]))

decoded = ""
r, c = 0, 0
for _ in range(cols):
    if r == 0:
        down = True
    elif r == rails - 1:
        down = False
    decoded += m[r][c]
    c += 1
    if down:
        r += 1
    else:
        r -= 1
print(f"Flag: {decoded}")

def problem3():
    """Reference:
    https://www.geeksforgeeks.org/dsa/rail-fence-cipher-encryption-decryption/
    """
    print("Problem: Rail Force One")
    ciphertext = "MHASIDYAKHBEWDEMFXTNURRCQYTVORSRYAILEIHZTLTGOSREE4EUJATBOFO"
    rails = 16
    rail_fence_decode(rails, ciphertext)

def problem4():
    """Reference:
    https://www.geeksforgeeks.org/dsa/rail-fence-cipher-encryption-decryption/
    """
    print("Problem: Rail Force Two")
    ciphertext = "MAYBETRYRAILS4AFTERTHISTCNMEASHIKWFUPVRLZGEUBOOJEOTEYOQRXDHD"[23:]
    rail_fence_decode(4, ciphertext)

if __name__ == "__main__":
    n = sys.argv
    if len(n) != 2:
        print(f"Usage solution.py <problem_id>")

```

```
    exit()  
prob = int(n[1])  
exec(f"problem{prob}()")
```